

Pattern Matching in Pizarnik

V. E. McHale

June 7, 2025

Contents

1	Introduction	1
2	Extensible Pattern Matches	1
2.1	Reusing Pattern-Match Arms	2
2.2	Typesafe Head	3
2.3	Pattern Terms	3

1 Introduction

Pizarnik’s approach to pattern-matching is based on commitment to the idea of pattern-matching as inverse[2] and suggestive notation from linear logic ($\&$, “with”, is dual to disjunction, $\oplus$) [5].

The actual machinery for extensible cases has already been accomplished by Blume, Acar, and Chae [1] with the end of solving the expression problem.

2 Extensible Pattern Matches

Suppose we define an algebraic data type

```
type Bool = `true  $\otimes$  `false;
```

The literal ``true` pushes a tag<sup>1</sup> ``true` onto the stack. We assign it the type `-- `true`. Its inverse ``true-1` pops a value of type ``true` off the stack and hence the atom ``true-1` has type ``true --` (inverse exchanges left and right).

In linear logic, one has $(G \oplus H)^\perp = G^\perp \& H^\perp$. To pop off a value of type `Bool = { `true $\otimes$ `false }`, one writes `{ `true-1 & `false-1 }` in Pizarnik. Or-patterns in other programming languages work like linear logic’s $\&$, which identifies a product of deconstructors with a deconstructor for a sum—the De Morgan laws.²

¹Inspired by symbols (e.g. ``blue`) in `k`, which are preceded by symbols (`'red`) in `Lisp`.

²The duality is worth noting: if we have two choices for returning a value of type $A \oplus B$ then a function accepting $A \oplus B$ must *provide* two pattern match arms.

Taking `not : Bool -- Bool`, we say

``true not : -- Bool`

``true` has type `-- `true` which can be generalized to `-- `true ⊕ `false` (an acceptable argument to `not : Bool -- Bool`). Sum types generalize but *only on the right*. Compare the situation in linear logic [4][p. 72]

$$\frac{\vdash \Gamma, A \quad \vdash \Gamma, B}{\vdash \Gamma, A \& B} \quad \frac{\vdash \Gamma, A}{\vdash \Gamma, A \oplus B} \quad \frac{\vdash \Gamma, B}{\vdash \Gamma, A \oplus B}$$

Forbidding $\&$ -generalization on the right manifests as forbidding inexhaustive pattern-matches. $\oplus$ is a disjunction; $\&$ is a conjunction; it restricts.³

Recalling the identity rule⁴.

$$\overline{\vdash A, A^\perp} \quad (\text{ID})$$

and the definition of $(-)^{\perp}$ by the De Morgan laws $(A \oplus B)^{\perp} = A^{\perp} \& B^{\perp}$ [3][p. 20], we can prove that $\&$ (with) generalizes on the left. And indeed a function accepting a value of type $A \oplus B$ as argument can just as well accept a value of type A , though this is not manifest in, say, Haskell. But it turns out to be useful; we define `foldr` to work on non-empty lists in §2.2.

2.1 Reusing Pattern-Match Arms

We can define a function

```
if : a b `true -- a
    := [ { `true-1 drop } ]
```

This is not so useful on its own—it requires the programmer to supply exactly one value and then discards it—but see how it unifies:

```
else : a b `false -- b
      := [ { `false-1 nip } ]
```

```
choice : a a Bool -- a
        := [ { if & else } ]
```

Pattern-match arms are terms like builtin functions (`+`, `swap`, etc.) and can be reused (named); one enjoys cut elimination for sequences of such.

³ $\&$ is a product; it would be farcical for a programmer to supply 1 when a term of type $(\text{Int} * \text{Int})$ is required. Yet this is exactly what allowing inexhaustive and redundant pattern matches encourages!

⁴This pairs well with the fact that inverses exchange left and right, as pointed out by Ehrenberg[2]

2.2 Typesafe Head

```
type List a = { `nil @ List(a) a `cons };

type NE a = { List(a) a `cons };

head : NE a -- a
      := [ { `cons-1 nip } ]
```

Any nonempty list can be supplied as an argument to a function on lists. This is far more satisfactory than the situation in Haskell, in which `foldr` &c. must be defined for `NonEmpty` and `List` separately.⁵

2.3 Pattern Terms

Suppose we have

```
type Ord = { `lt @ `eq @ `gt };
```

Then we can define

```
lte : { `lt @ `eq } --
      := [ { `lt-1 & `eq-1 } ]

gt : Ord -- Bool
      := [ { lte `false & `gt-1 `true } ]
```

So or-patterns are reusable, enjoy cut-elimination, etc.

References

- [1] Matthias Blume, Umut A. Acar, and Wonseok Chae. Extensible programming with first-class cases. *SIGPLAN Not.*, 41(9):239–250, sep 2006.
- [2] Daniel Ehrenberg. Pattern matching in concatenative programming languages. 2009.
- [3] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.
- [4] Jean-Yves Girard. Linear logic: A survey. In Friedrich L. Bauer, Wilfried Brauer, and Helmut Schwichtenberg, editors, *Logic and Algebra of Specification*, pages 63–112, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [5] Guillaume Munch-Maccagnoni. Focalisation and classical realisability. In Erich Grädel and Reinhard Kahle, editors, *Computer Science Logic*, pages 409–423, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

⁵Typeclasses allow `foldr` to be called on non-empty lists and lists equally but instances must be defined separately, and the typeclass machinery brings globality of instances and other complications.